

UNIwersYTET RZESZOWSKI
WYDZIAŁ NAUK ŚCISŁYCH I TECHNICZNYCH
INSTYTUT INFORMATYKI



Filip Lisowski
134942

Informatyka

Aplikacja REPAIRO - dokumentacja projektowa

Praca projektowa

Praca wykonana pod kierunkiem
mgr inż. Ewa Żesławska

Rzeszów 2025

Spis treści

1. Wprowadzenie	7
2. Opis założeń projektu	8
2.1. Cel projektu	8
2.1.1. Analiza problemu	8
2.1.2. Plan realizacji projektu	8
2.2. Wymagania funkcjonalne	9
2.2.1. Logowanie do systemu	9
2.2.2. Funkcjonalność dla roli administrator i menadżer	9
2.2.3. Funkcjonalność dla roli mechanik	10
2.2.4. Funkcjonalność dla roli recepcjonista	10
2.2.5. Funkcjonalność dla roli magazynier	10
2.2.6. Porównanie ról użytkowników i ich uprawnień	11
2.3. Wymagania niefunkcjonalne	11
2.3.1. Środowisko testowe	11
2.3.2. Wydajność	12
2.3.3. Bezpieczeństwo	12
2.3.4. Użyteczność	13
2.3.5. Skalowalność	13
2.3.6. Integralność danych	14
3. Opis struktury projektu	15
3.1. Technologie wykorzystane w projekcie	15
3.2. Struktura katalogów i plików	16
3.3. Architektura aplikacji i hierarchia klas	17
3.3.1. Architektura systemu	17
3.3.2. Hierarchia pakietów	17
3.3.3. Przepływ danych w aplikacji	17
3.3.4. Wybrane klasy i ich odpowiedzialności	17
3.3.5. Zasada pojedynczej odpowiedzialności (SRP)	18
3.3.6. Walidacja danych	18
3.3.7. Diagram UML	19
3.4. Pakiet resources - widoki graficzne i zasoby pomocnicze	20
3.4.1. Pliki FXML - definicje widoków	20
3.4.2. Pozostałe zasoby	20
3.5. Zarządzanie danymi i bazą danych	21
3.5.1. Struktura encji i relacje	21
3.5.2. Opis przykładowych tabel	21

3.5.3. Relacje między tabelami	22
3.5.4. Podsumowanie	22
4. Harmonogram realizacji projektu	23
4.1. Etapy realizacji oraz wykres Gantta	23
4.2. System kontroli wersji i repozytorium projektu	23
5. Prezentacja warstwy użytkowej projektu.....	24
5.1. Opis szaty graficznej.....	24
5.2. Prezentacja głównych widoków interfejsu użytkownika	24
5.2.1. Okno logowania	24
5.2.2. Główne okno interfejsu.....	25
5.2.3. Okno zarządzania samochodami.....	25
5.2.4. Okno zarządzania klientami.....	26
5.2.5. Okno zarządzania przyjęciami.....	26
5.2.6. Okno zarządzania częściami	27
5.2.7. Okno zarządzania wizytami	27
5.2.8. Okno zarządzania pracownikami	28
5.2.9. Okno zarządzania usługami	28
5.2.10. Okno wyświetlania informacji o aplikacji	29
5.3. Mechanizm alertów systemowych.....	30
6. Podsumowanie	31
7. Instrukcja uruchomienia aplikacji REPAIRO	32
7.0.1. Wymagania środowiskowe.....	32
7.0.2. Baza danych	32
7.0.3. Konfiguracja połączenia z bazą danych	32
7.0.4. Kroki uruchomienia aplikacji REPAIRO.....	33
7.0.5. Dane testowe (logowanie).....	34
Spis rysunków	35
Spis tabel	36
Oświadczenie studenta o samodzielności pracy	37

1. Wprowadzenie

REPAIRO to aplikacja desktopowa, zaprojektowana do zarządzania serwisem samochodowym. Powstała w języku Java z wykorzystaniem biblioteki JavaFX oraz relacyjnej bazy danych MySQL. System umożliwia obsługę procesów związanych z funkcjonowaniem warsztatu: rejestrację klientów i pojazdów, tworzenie zleceń napraw, zarządzanie usługami i częściami oraz przypisywanie pracowników do konkretnych zadań.

Interfejs użytkownika został zaprojektowany w duchu przejrzystości i intuicyjności - wszystkie funkcjonalności dostępne są z poziomu czytelnych formularzy, a dane wprowadzane przez użytkownika są walidowane, co ogranicza ryzyko błędów. System automatycznie przelicza koszt zlecenia na podstawie wybranych usług i części, wspierając dokładność wycen oraz organizację pracy.

Aplikacja może być wykorzystywana zarówno w małych, jak i średnich warsztatach, gdzie potrzeba systematycznego zarządzania danymi klientów i pojazdów staje się coraz bardziej istotna.

REPAIRO is a desktop application designed to manage a car service. It was created in Java using the JavaFX library and the MySQL relational database. The system enables the handling of processes related to the functioning of the workshop: registration of customers and vehicles, creation of repair orders, management of services and parts, and assignment of employees to specific tasks.

The user interface was designed in the spirit of transparency and intuitiveness - all functionalities are available from the level of legible forms, and data entered by the user is validated, which reduces the risk of errors. The system automatically calculates the cost of the order based on selected services and parts, supporting the accuracy of pricing and work organization.

The application can be used in both small and medium-sized workshops, where the need for systematic management of customer and vehicle data is becoming increasingly important.

2. Opis założeń projektu

2.1. Cel projektu

Celem projektu jest stworzenie funkcjonalnego, przejrzystego i intuicyjnego systemu, który wspiera zarządzanie warsztatem samochodowym. Obejmuje to:

- dodawanie danych klientów oraz ich pojazdów,
- tworzenie zleceń napraw z przypisanymi usługami, częściami i pracownikami,
- kontrolę i zarządzanie dostępnymi częściami i usługami,
- przeliczanie kosztów zlecenia na podstawie wybranych pozycji,
- zapewnienie prostego i przejrzystego interfejsu użytkownika.

2.1.1. Analiza problemu

Zarządzanie warsztatem samochodowym to złożony proces, wymagający sprawnej organizacji informacji oraz płynnej obsługi klientów. Wiele warsztatów wciąż operuje na papierowej dokumentacji lub w niedostosowanych narzędziach ogólnego przeznaczenia, co wpływa negatywnie na efektywność pracy. Projekt ten odpowiada na realne potrzeby branży mechanicznej, oferując cyfrowe rozwiązanie dostosowane do pracy serwisu. Problem ten wpływa bezpośrednio na efektywność pracy warsztatu oraz przejrzystość informacji. Istnieje zapotrzebowanie na prosty, ale funkcjonalny system, który umożliwiłby szybkie dodawanie zleceń, przypisywanie ich do konkretnych pracowników i klientów oraz kontrolowanie terminów i kosztów usług.

2.1.2. Plan realizacji projektu

Projekt zostanie wdrażany etapami, zgodnie z poniższą strukturą:

1. Wstępna analiza potrzeb użytkowników oraz określenie kluczowych funkcjonalności.
2. Stworzenie projektu relacyjnej bazy danych (MySQL), gwarantującej spójność i wydajność danych,
3. Implementacja interfejsu graficznego z użyciem technologii JavaFX,
4. Integracja logiki aplikacji z bazą danych,
5. Przeprowadzenie testów w celu wyeliminowania błędów,
6. Sporządzenie kompletnej dokumentacji obejmującej założenia projektu, opis architektury oraz instrukcję obsługi.

Efektem finalnym będzie funkcjonalna aplikacja desktopowa wspomagająca działalność warsztatu samochodowego. Dodatkowo, system będzie posiadał zabezpieczenia oraz możliwość rozbudowy o dodatkowe funkcjonalności w przyszłości.

2.2. Wymagania funkcjonalne

System zarządzania serwisem samochodowym REPAIRO pozwala użytkownikom realizację konkretnych zadań w zależności od przypisanej roli. Każdy użytkownik, po zalogowaniu, ma dostęp wyłącznie do funkcjonalności niezbędnych do wykonywania swoich obowiązków. W tej sekcji opisano wymagania funkcjonalne procesu logowania a następnie przedstawiono podział funkcji dostępnych w aplikacji w zależności od roli użytkownika.

2.2.1. Logowanie do systemu

- System umożliwia logowanie się do aplikacji i zapewnia dostęp wyłącznie uprawnionym użytkownikom. Uwierzytelnianie odbywa się na podstawie danych przechowywanych w bazie danych - użytkownik wprowadza nazwę użytkownika i hasło, a system sprawdza poprawność tych danych. W celu zapewnienia bezpieczeństwa, pole hasło domyślnie jest ukryte, z możliwością jego odsłonięcia za pomocą przycisku.
- System wyposażono w mechanizmy ochronne. W przypadku wykrycia nieznanej roli, użytkownik otrzymuje jednoznaczny komunikat o błędzie. Przy próbie logowania z błędnymi danymi (np. nieistniejący użytkownik lub niepoprawne hasło), aplikacja wyświetla odpowiedni komunikat.
- W przypadku udanego logowania, system ustala rolę użytkownika (np. administrator, menadżer, mechanik, recepcjonista, magazynier) oraz pobiera dane z relacyjnych tabel bazy danych, takie jak pełna nazwa stanowiska oraz imię użytkownika, które są następnie wyświetlane na panelu głównym aplikacji. Interfejs dostosowuje się do poziomu uprawnień każdej z ról, wyświetlając tylko dostępne widoki.
- Po zalogowaniu, dane użytkownika - jego imię, stanowisko oraz rola - są przechowywane w pamięci aplikacji aż do momentu jej zamknięcia.

Po udanym zalogowaniu, użytkownik zostaje przekierowany do panelu głównego, który jest dostosowany do jego roli. Każda rola ma przypisane odpowiednie funkcje związane z kompetencjami i obowiązkami użytkowników. Niezależnie od roli, każdy użytkownik ma dostęp do wspólnych widoków systemowych, takich jak ekran logowania, wylogowanie się czy widok informacji o aplikacji. Użytkownik nie widzi zakładki i widoków, do których nie ma przypisanych uprawnień. Poniżej opisano szczegółowo funkcje dostępne dla pięciu ról w systemie: administratora, menadżera, mechanika, recepcjonisty i magazyniera.

2.2.2. Funkcjonalność dla roli administrator i menadżer

Użytkownicy z rolą **administratora i menadżera** posiadają najwyższy stopień uprawnień w systemie. Logują się do systemu, wprowadzając unikalny login oraz hasło. Po zalogowaniu, mają dostęp do interfejsu, który umożliwia mu zarządzanie kluczowymi opcjami:

- przeglądać, dodawać i usuwać pojazdy,
- przeglądać, dodawać i usuwać klientów,
- przeglądać, dodawać i usuwać części oraz usługi,
- przeglądać listę pracowników (na potrzeby projektu założono, że zarządzanie pracownikami odbywa się wyłącznie poprzez system bazodanowy),
- przeglądać, dodawać, edytować i usuwać zlecenia napraw,

- wyświetlać kalendarz wizyt, w którym prezentowane są aktywne zlecenia.

Administator ma również możliwość swobodnego przełączania się pomiędzy widokami aplikacji, co pozwala na łatwy dostęp do wszystkich funkcjonalności.

2.2.3. Funkcjonalność dla roli mechanik

Użytkownicy z rolą **mechanika** logują się do systemu za pomocą unikalnego loginu i hasła. Po zalogowaniu, mają dostęp do interfejsu, który umożliwia mu zarządzanie kluczowymi opcjami.

Mechanik ma dostęp do następujących funkcji:

- przeglądać, dodawać i usuwać pojazdy,
- przeglądać, dodawać i usuwać części oraz usługi,
- przeglądać, dodawać, edytować i usuwać zlecenia napraw,
- korzystania z dedykowanego widoku umożliwiającego sprawne przełączanie się między zakładkami i sekcjami aplikacji.

Interfejs graficzny został dostosowany do potrzeb pracownika technicznego - ograniczony do niezbędnych funkcjonalności, co upraszcza obsługę systemu.

2.2.4. Funkcjonalność dla roli recepcjonista

Użytkownicy z rolą **recepcjonisty** logują się do systemu za pomocą unikalnego loginu i hasła. Po zalogowaniu, mają dostęp do interfejsu, który umożliwia mu zarządzanie kluczowymi opcjami.

Recepcjonista ma dostęp do następujących funkcji:

- przeglądać, dodawać i usuwać pojazdy,
- przeglądać, dodawać i usuwać klientów,
- przeglądać, dodawać, edytować i usuwać zlecenia napraw,
- przeglądanie kalendarza wizyt w celu koordynacji pracy warsztatu.

Interfejs graficzny został dostosowany do potrzeb pracownika obsługi klienta - ograniczony do niezbędnych funkcjonalności, co upraszcza obsługę systemu.

2.2.5. Funkcjonalność dla roli magazynier

Użytkownicy z rolą **magazyniera** logują się do systemu za pomocą unikalnego loginu i hasła. Po zalogowaniu, mają dostęp do interfejsu, który umożliwia mu zarządzanie kluczowymi opcjami.

Rola magazyniera ogranicza się jedynie do zarządzania częściami samochodowymi. W związku z tym, użytkownik ma dostęp jedynie do widoku części, gdzie może:

- dodawać nowe części do systemu,
- przeglądać dostępne części,
- usuwać części, które nie są już potrzebne.

Interfejs graficzny został dostosowany do potrzeb pracownika magazynu - ograniczony do niezbędnych funkcjonalności, co upraszcza obsługę systemu.

Wszystkie role użytkowników mają również dostęp do wspólnych widoków systemowych, takich jak: ekran logowania, informacje o aplikacji, możliwość zmiany hasła oraz wylogowania się. Przełączanie między widokami jest dostępne zgodnie z przydzielonymi uprawnieniami - użytkownik widzi jedynie te opcje, do których posiada dostęp.

2.2.6. Porównanie ról użytkowników i ich uprawnień

Poniżej przedstawiono tabelę porównującą dostęp poszczególnych ról użytkowników do modułów systemu. Znakiem ✓ oznaczono pełny dostęp, natomiast znakiem ✗ brak dostępu:

Tabela 2.1. Dostęp poszczególnych ról użytkowników do modułów aplikacji.

Moduł	Administrator	Manager	Mechanik	Recepcjonista	Magazynier
Samochody	✓	✓	✓	✓	✗
Klienci	✓	✓	✗	✓	✗
Przyjęcia	✓	✓	✓	✓	✗
Części	✓	✓	✓	✗	✓
Wizyty (kalendarz)	✓	✓	✗	✓	✗
Pracownicy (podgląd)	✓	✓	✗	✗	✗
Usługi	✓	✓	✓	✗	✗
O aplikacji	✓	✓	✓	✓	✓

2.3. Wymagania niefunkcjonalne

2.3.1. Środowisko testowe

Testy aplikacji przeprowadzono w środowisku o konfiguracji:

- Procesor: Intel Core i5-13500
- Karta graficzna: NVidia GeForce RTX 3060
- Pamięć RAM: 32 GB
- Dysk: SSD NVMe 1024 GB
- System operacyjny: Windows 11 Home
- IDE: IntelliJ IDEA 2024.3.3
- JDK: OpenJDK 23.0.2
- JavaFX: 23.0.1 (zgodnie z deklaracją xmlns)
- JDBC: mysql-connector-java-8.0.33

- Baza danych: MySQL (phpMyAdmin)
 - Serwer: 127.0.0.1 via TCP/IP
 - Typ serwera: MariaDB
 - Połączenie z serwerem: SSL nie jest używany
 - Wersja serwera: 10.4.32-MariaDB - mariadb.org binary distribution
 - Wersja protokołu: 10
 - Użytkownik: root@localhost
 - Kodowanie znaków serwera: UTF-8 Unicode (utf8mb4)

2.3.2. Wydajność

Aplikacja została zoptymalizowana pod kątem wydajności, aby zapewnić płynne działanie nawet przy dużej liczbie danych. Przeprowadzone testy wykazały:

- Czas ładowania aplikacji wynosi średnio 2 sekundy.
- Interfejs użytkownika jest responsywny, a przełączanie między widokami odbywa się bez opóźnień.
- Operacje na bazie danych (dodawanie, edytowanie, usuwanie rekordów) są realizowane w czasie nieprzekraczającym 1 sekundy dla pojedynczych rekordów.
- W trakcie działania aplikacja zużywa średnio 350 MB pamięci RAM oraz nie obciąża procesora.

2.3.3. Bezpieczeństwo

Aplikacja została wyposażona w mechanizmy zabezpieczeń, które mają na celu zapewnienie integralności i dostępności informacji.

- Aplikacja za każdym razem wymaga logowania przed uzyskaniem dostępu do systemu.
- Każdy użytkownik posiada unikalną nazwę użytkownika,
- Pole hasła zostało ukryte, a użytkownik ma możliwość odsłonięcia go za pomocą przycisku.
- Uprawnienia użytkowników są kontrolowane na podstawie roli użytkownika,
- Formularze posiadają walidację danych - nie można zapisać pustych ani niepoprawnych wartości (np. litery w polu liczbowym),
- Komunikaty informujące o sukcesie lub błędzie wyświetlają jasną i klarowną użytkownikowi wiadomość,
- Pola wymagające powiązania z innymi rekordami (np. pojazd, klient) są wybierane z rozwijanych list, co eliminuje możliwość wpisania niepoprawnych danych.

Aplikacja została stworzona z myślą o dalszym rozwoju i może być łatwo rozszerzona o dodatkowe zabezpieczenia, takie jak szyfrowanie danych czy logowanie zdarzeń.

2.3.4. Użyteczność

Użyteczność systemu odgrywa kluczową rolę w zapewnieniu efektywnej i przyjaznej obsługi przez użytkowników. Poniższe punkty definiują podstawowe założenia dotyczące wygody i intuicyjności korzystania z aplikacji.

- Czytelny oraz intuicyjny interfejs z jasno opisnymi przyciskami i polami edycji.
- Formularze z walidacją danych - użytkownik nie może wprowadzić błędnych danych, a system informuje o błędach w czasie rzeczywistym.
- Informacje zwrotne po wykonaniu operacji o sukcesie lub błędzie wyświetlane w formie komunikatów wyświetlane są natychmiast.
- Usuwanie rekordów poprzedzone jest potwierdzeniem, aby uniknąć przypadkowego usunięcia danych.
- Responsywność interfejsu - aplikacja działa płynnie, a przełączanie między widokami nie powoduje opóźnień.
- Użycie ComboBoxów i kalendarzy (DatePicker) do wyboru danych zamiast ręcznego wpisywania.
- Obsługa różnych ról użytkowników z dostosowanym widokiem (np. mechanik widzi inne funkcje niż administrator).

Przestrzeganie tych zasad ma na celu zwiększenie satysfakcji użytkowników oraz minimalizację błędów wynikających z nieporozumień lub trudności obsługi systemu.

2.3.5. Skalowalność

Aplikacja została zaprojektowana z myślą o przyszłej rozbudowie i skalowalności. Poniższe aspekty zapewniają elastyczność i możliwość dostosowania systemu do rosnących potrzeb:

- Możliwość dodania hashowania haseł w celu zapewnienia bezpieczeństwa danych użytkowników.
- Możliwość dodania dynamicznego zmieniania rozmiaru okna (powiększanie, pomniejszanie) poprawia użyteczność i pozwala na łatwiejszą adaptację interfejsu do różnych ekranów.
- Struktura bazy danych została zaprojektowana z myślą o rozbudowie, umożliwiającą dodawanie nowych tabel i relacji bez zaburzania działania istniejących funkcji.
- Interfejs aplikacji jest modularny i prosty do rozbudowy, co pozwala łatwo dodawać nowe zakładki i funkcjonalności bez przebudowy całego systemu.
- Możliwość rozbudowania aplikacji o integrację z innymi systemami, np. pocztą elektroniczną czy systemami płatności.
- Struktura kodu i organizacja plików (jasny podział na warstwy, foldery według funkcjonalności i typów komponentów) ułatwiają orientację i rozwój aplikacji przez kolejnych programistów.
- Obecny model przypisywania części do zleceń jest ograniczony do jednego rodzaju części, jednak architektura pozwala na łatwe rozszerzenie tej funkcjonalności, by obsłużyć wiele części dla jednego zlecenia.

Zorganizowana struktura kodu oraz bazy danych umożliwia łatwe wprowadzanie zmian i dodawanie nowych funkcji, co jest kluczowe w kontekście dynamicznie zmieniających się potrzeb użytkowników i rynku.

2.3.6. Integralność danych

Integralność danych gwarantuje, że informacje przechowywane i przetwarzane w systemie są spójne, poprawne i wiarygodne przez cały czas działania aplikacji.

- **Klucze główne i obce:** - odpowiednio zdefiniowane relacje między tabelami w bazie danych zapobiegają powstawaniu niespójnych rekordów.
- **Walidacja danych na poziomie aplikacji** - aplikacja sprawdza poprawność wprowadzanych danych przed ich zapisaniem do bazy, co minimalizuje ryzyko błędów.
- **Spójność danych wprowadzanych przez użytkownika** - system wymaga uzupełnienia wszystkich niezbędnych informacji, eliminując możliwość pozostawienia pustych lub błędnych pól.
- **Obsługa błędów** - system odpowiednio reaguje na błędy podczas operacji na danych, wyświetlając użytkownikowi klarowne komunikaty o sukcesie lub porażce.

Zastosowanie tych rozwiązań gwarantuje spójność danych w systemie oraz minimalizuje ryzyko błędów, które mogłyby negatywnie wpłynąć na funkcjonowanie aplikacji.

3. Opis struktury projektu

W tym rozdziale przedstawiona zostanie ogólna struktura projektu, ze szczególnym uwzględnieniem wykorzystanych technologii, organizacji kodu oraz zarządzania danymi. Opisane zostaną hierarchie klas, najważniejsze metody oraz wymagania sprzętowe niezbędne do uruchomienia aplikacji.

3.1. Technologie wykorzystane w projekcie

W projekcie wykorzystano szereg technologii umożliwiających tworzenie aplikacji typu desktop z graficznym interfejsem użytkownika, komunikację z relacyjną bazą danych oraz zapewnienie odpowiedniego zarządzania zależnościami i strukturą kodu.

- **Java (OpenJDK 23.0.2)** - język programowania, w którym została napisana aplikacja.
- **JavaFX 23.0.1** - biblioteka graficzna do tworzenia interfejsów użytkownika.
- **FXML + SceneBuilder** - wszystkie widoki interfejsu użytkownika zostały zaprojektowane w Scene Builderze i zapisane jako pliki FXML, co umożliwia rozdzielenie widoków od logiki.
- **JDBC (Java Database Connectivity)** - API służące do komunikacji z relacyjną bazą danych.
- **MySQL Connector/J 8.0.33** - biblioteka umożliwiająca komunikację aplikacji Java z bazą danych MySQL.
- **MySQL** - relacyjna baza danych przechowująca dane dotyczące klientów, pojazdów, zleceń, pracowników itd.
- **phpMyAdmin** - webowe narzędzie do graficznego zarządzania bazą danych MySQL: tworzenia tabel, relacji, testowania zapytań i modyfikacji danych.
- **Maven 3.8.5** - system zarządzania projektem i zależnościami, umożliwiający automatyczne pobieranie bibliotek i budowanie aplikacji.
- **IDE: IntelliJ IDEA 2024.3.3** - środowisko programistyczne używane do pisania kodu, konfiguracji projektu oraz integracji.

3.2. Struktura katalogów i plików

Poniższa struktura katalogów odzwierciedla typowy układ projektu Java z wykorzystaniem narzędzi Maven oraz frameworka JavaFX. Główne foldery zostały podzielone zgodnie z zasadą separacji odpowiedzialności, co ułatwia zarządzanie kodem oraz jego dalszą rozbudowę.

- `src/main/java/project.app` - zawiera kod źródłowy aplikacji podzielony na pakiety zgodnie z rolami komponentów: kontrolery JavaFX (**controllers**), warstwa dostępu do danych (**dao**), modele danych (**model**), interfejsy (**interfaces**) oraz narzędzia pomocnicze (**utils**).
- `src/main/resources` - miejsce na zasoby statyczne, takie jak widoki interfejsu użytkownika w formacie FXML (**project.app**), pliki stylów CSS (**values**), obrazy (**images**) oraz plik konfiguracyjny połączenia z bazą danych (**config.properties**).
- Plik na poziomie głównym, taki jak **pom.xml**, odpowiada za konfigurację projektu Maven, a dokumenty **README.md** i **CHANGELOG.md** zawierają opis projektu oraz historię zmian.

```
|-- SystemZarządzaniaSerwisemSamochodowym/
|   |-- src/
|       |-- main/
|           |-- java/
|               |-- project.app/
|                   |-- controllers/      <- Kontrolery JavaFX
|                   |-- dao/             <- Dostęp do danych (JDBC)
|                   |-- interfaces/      <- Interfejsy
|                   |-- model/           <- Modele danych
|                   |-- utils/           <- Klasy pomocnicze
|                   |-- Main.java        <- Klasa główna
|                   |-- module-info.java
|               |-- resources/
|                   |-- images/          <- Zasoby graficzne
|                   |-- project.app/     <- Widoki FXML
|                   |-- values/          <- Pliki CSS
|                   |-- config.properties <- Plik konfiguracyjny bazy danych
|-- pom.xml
|-- CHANGELOG.md
|-- README.md
```

Rys. 3.1. Struktura katalogów projektu

3.3. Architektura aplikacji i hierarchia klas

3.3.1. Architektura systemu

Aplikacja została zaprojektowana zgodnie z architekturą trójwarstwową, obejmującą warstwy: prezentacji, logiki oraz dostępu do danych. Podział ten sprzyja modularności i ułatwia testowanie oraz rozwój aplikacji.

- **Warstwa prezentacji** - realizowana w JavaFX, odpowiada za interakcję z użytkownikiem.
- **Warstwa logiki** - zawiera klasy pośredniczące pomiędzy GUI a bazą danych.
- **Warstwa dostępu do danych (DAO)** - odpowiada za komunikację z bazą danych przy użyciu JDBC.

3.3.2. Hierarchia pakietów

Struktura pakietów w projekcie jest następująca:

```
project.app/  
|-- controllers/      <- Kontrolery JavaFX  
|-- dao/              <- Warstwa dostępu do danych  
|-- interfaces/       <- Interfejsy  
|-- model/            <- Modele danych  
|-- utils/            <- Klasy pomocnicze  
`- Main.java          <- Punkt wejścia do aplikacji
```

Rys. 3.2. Struktura pakietów

3.3.3. Przepływ danych w aplikacji

Poniżej przedstawiono przykładowy przepływ danych w aplikacji na przykładzie dodawania nowego klienta:

1. Użytkownik wprowadza dane klienta w formularzu JavaFX (`CustomerController`).
2. Dane są walidowane i umieszczane w obiekcie modelu typu `Customer`.
3. Następnie kontroler wywołuje metodę `CustomerDAO.add(Customer customer)`, odpowiedzialną za zapis danych do bazy.
4. Po pomyślnym dodaniu rekordu, kontroler odświeża widok tabeli.

3.3.4. Wybrane klasy i ich odpowiedzialności

W niniejszym rozdziale opisano wybrane klasy reprezentujące trzy główne warstwy architektury aplikacji: prezentacji, logiki biznesowej oraz dostępu do danych. Przedstawione klasy zostały wybrane jako reprezentatywne przykłady, ukazujące schemat projektowy zastosowany w całym systemie. Pozostałe komponenty aplikacji zachowują analogiczną strukturę oraz sposób działania — każda warstwa została zaprojektowana zgodnie ze spójnym wzorcem, co zapewnia czytelność, łatwość konserwacji oraz możliwość dalszej rozbudowy systemu.

Warstwa prezentacji

- **CarsController** - kontroler widoku zarządzania pojazdami. Odpowiada za wczytywanie danych z bazy (marki, modele, typy nadwozi itp.) do rozwijanych list, obsługę formularza oraz synchronizację danych z tabelą.
- **OrdersController** - obsługuje tworzenie i edycję zleceń. Pobiera dane z wielu źródeł (klient, pojazd, usługa, pracownik) i integruje je w jednym widoku. Zapewnia walidację oraz komunikację z odpowiednimi DAO.

Warstwa logiki (model)

- **Order** - model reprezentujący zlecenie. Zawiera referencje do powiązanych obiektów: klienta, pojazdu, usługi, pracownika, a także informacje o dacie zakończenia, statusie i koszcie.
- **Employee** - reprezentuje pracownika serwisu. Zawiera dane identyfikacyjne oraz typ konta (rola), służące m.in. do logowania oraz przypisywania zleceń.

Warstwa dostępu do danych (DAO)

- **CustomerDAO** - klasa odpowiedzialna za operacje CRUD na tabeli klientów. Wykorzystuje `PreparedStatement` do bezpiecznego wykonywania zapytań SQL.
- **VehicleDAO** - zapewnia obsługę operacji na pojazdach: dodawanie, edycję, pobieranie danych do ComboBoxów oraz usuwanie rekordów.

3.3.5. Zasada pojedynczej odpowiedzialności (SRP)

W projekcie stosowana jest zasada pojedynczej odpowiedzialności (Single Responsibility Principle). Każda klasa odpowiada za jeden, jasno określony aspekt działania systemu. Przykładowo, `CustomerController` odpowiada wyłącznie za interakcję z użytkownikiem, a `CustomerDAO` – za operacje na danych klientów w bazie. Taki podział ułatwia rozwój oraz testowanie poszczególnych komponentów.

3.3.6. Walidacja danych

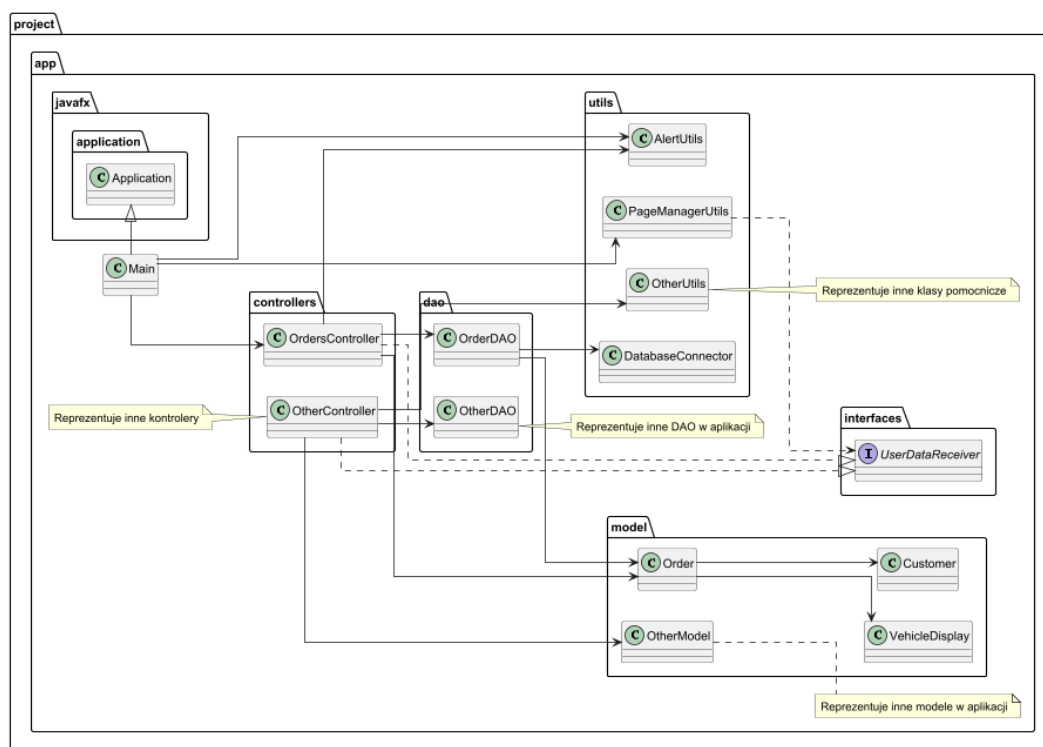
Dane wprowadzane przez użytkownika są walidowane po stronie kontrolera, jeszcze przed próbą zapisu do bazy danych. Weryfikowana jest m.in. kompletność pól oraz poprawność formatu (np. numeru telefonu, liczby znaków). W przypadku wykrycia nieprawidłowości, użytkownik otrzymuje komunikat o błędzie.

3.3.7. Diagram UML

Na rysunku (rys. 3.3) przedstawiono uproszczony diagram klas aplikacji. Diagram ten ilustruje sposób komunikacji między warstwami aplikacji oraz najważniejsze relacje pomiędzy reprezentatywnymi klasami. W celu zwiększenia czytelności i przejrzystości diagramu zastosowano następujące uproszczenia:

- Zamiast przedstawiania wszystkich klas w projekcie, pozostawiono po jednym reprezentatywnym przykładzie z każdego głównego pakietu, takich jak `dao`, `model`, `utils` oraz `controllers`.
- Dodatkowo, dla zachowania kontekstu architektury, umieszczono ogólne klasy pomocnicze, takie jak `OtherDAO`, `OtherModel`, `OtherController` oraz `OtherUtils`, symbolizujące pozostałe komponenty danej warstwy.
- Pokazano dziedziczenie klasy `Main` po `javafx.application.Application`, aby podkreślić związek z technologią JavaFX.
- Zachowano interfejs `UserDataReceiver`, który implementowany jest przez kontrolery, co pokazuje wspólny kontrakt danych użytkownika.

Taka forma diagramu pozwala na zrozumienie ogólnej architektury systemu bez nadmiernego zagłębiania się w szczegóły implementacyjne.



Rys. 3.3. Diagram UML

3.4. Pakiet resources - widoki graficzne i zasoby pomocnicze

Pakiet `resources` zawiera wszystkie pliki niezbędne do prezentacji graficznej interfejsu użytkownika, a także dodatkowe zasoby wspierające działanie aplikacji. Struktura tego katalogu została podzielona na trzy główne grupy: pliki FXML, style CSS oraz multimedia (opcjonalne).

3.4.1. Pliki FXML - definicje widoków

Pliki z rozszerzeniem `.fxml` definiują strukturę graficznych interfejsów użytkownika aplikacji w oparciu o technologię JavaFX. Każdy plik odpowiada za konkretny ekran lub widok w aplikacji. Zostały one przygotowane z użyciem narzędzia Scene Builder i połączone z odpowiednimi kontrolerami.

Plik FXML	Opis
<code>customersPage.fxml</code>	Widok do zarządzania klientami: formularz dodawania klienta, tabela klientów.
<code>carsPage.fxml</code>	Widok listy pojazdów oraz formularz dodawania nowych aut.
<code>ordersPage.fxml</code>	Widok zleceń - umożliwia dodawanie i przeglądanie zleceń warsztatowych.
<code>servicesPage.fxml</code>	Widok zarządzania usługami serwisowymi.
<code>employeesPage.fxml</code>	Widok wyświetlania pracowników serwisu.
<code>loginPage.fxml</code>	Ekran logowania do aplikacji.

Tabela 3.1. Pliki FXML i ich przeznaczenie

3.4.2. Pozostałe zasoby

W folderze `resources` mogą znajdować się także dodatkowe podkatalogi z multimediami:

- `values/` - pliki CSS definiujące styl graficzny aplikacji. Wpływają one na kolorystykę, wygląd przycisków, tabel oraz efektów interaktywnych.
- `images/` - ikony wykorzystywane w aplikacji.

Wszystkie ikony użyte w aplikacji pochodzą z serwisu Icons8. Tło oraz logo zostały opracowane na zamówienie.

3.5. Zarządzanie danymi i bazą danych

3.5.1. Struktura encji i relacje

W projekcie zastosowano relacyjną bazę danych, która przechowuje informacje niezbędne do zarządzania serwisem samochodowym. Główne encje (tabele) odzwierciedlają kluczowe obiekty systemu, takie jak: Klienci, Pojazdy, Pracownicy, Zlecenia, Usługi oraz Części.

Relacje między encjami są zaprojektowane zgodnie z zasadami normalizacji, aby uniknąć redundancji danych i zapewnić integralność. Przykładowo:

- Jeden klient może posiadać wiele pojazdów (relacja 1 do N).
- Każde zlecenie jest powiązane z dokładnie jednym pojazdem i jednym pracownikiem odpowiedzialnym za realizację.
- Usługi oraz części są przypisane do zleceń, co pozwala na precyzyjne rozliczenie kosztów naprawy.



Rys. 3.4. Diagram ERD bazy danych warsztat

3.5.2. Opis przykładowych tabel

- Klienci - Tabela zawiera dane klientów: imię, nazwisko, numer telefonu, adres e-mail, typ klienta (osoba prywatna lub firma) oraz, w przypadku firm, nazwę przedsiębiorstwa.
- Samochody - Przechowuje informacje o samochodach powiązanych z klientami, takie jak marka, model, typ nadwozia, rodzaj paliwa, pojemność silnika, napęd oraz przebieg.
- Pracownicy - Dane o pracownikach serwisu, w tym imię, nazwisko oraz stanowisko.
- Zlecenia - Rejestruje pojedyncze naprawy, zawierając powiązania do klienta, pojazdu, pracownika, usługi i części. Zawiera także informacje o statusie i terminie realizacji.

3.5.3. Relacje między tabelami

Struktura bazy danych opiera się na kluczach głównych (PK) i obcych (FK), które definiują powiązania między tabelami. Najważniejsze relacje to:

- Pojazdy (PK: car_id) → Zlecenia (FK: car_id),
- Pracownicy (PK: employee_id) → Zlecenia (FK: employee_id),
- Usługi oraz Części powiązane są z Zleceniami przez odpowiednie klucze,

Relacje zostały zaprojektowane tak, by zapewnić spójność i umożliwić łatwe rozszerzanie systemu.

3.5.4. Podsumowanie

Baza danych została zaprojektowana z myślą o przejrzystości i wydajności. Struktura encji i relacji gwarantuje integralność danych, minimalizuje ryzyko błędów oraz umożliwia sprawne zarządzanie informacjami w serwisie samochodowym. Takie podejście pozwala na łatwą rozbudowę funkcjonalności i utrzymanie systemu w przyszłości.

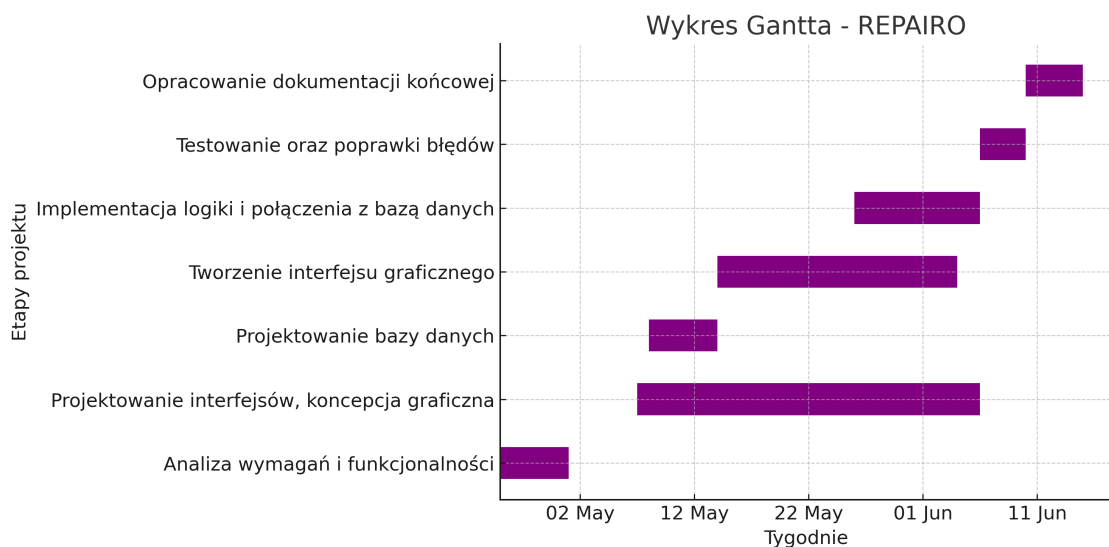
4. Harmonogram realizacji projektu

4.1. Etapy realizacji oraz wykres Gantta

Proces realizacji aplikacji do zarządzania serwisem samochodowym został podzielony na kilka logicznych etapów, obejmujących analizę wymagań funkcjonalnych, projektowanie graficzne oraz techniczne, implementację interfejsu użytkownika, połączenie z bazą danych, testowanie i końcowe opracowanie dokumentacji projektowej. Dzięki takiemu podziałowi możliwe było wdrażanie kolejnych funkcjonalności oraz szybka identyfikacja i eliminacja ewentualnych błędów na wczesnych etapach prac.

Realizacja przebiegała zgodnie z założonym planem. Niektóre zadania wymagały powrotu do wcześniejszych faz projektu - głównie w związku z wprowadzaniem nowych pomysłów, korektą koncepcji interfejsu lub dostosowaniem bazy danych do rozszerzonych wymagań funkcjonalnych.

Warto dodać, że niektóre działania były prowadzone równolegle, co pozwoliło usprawnić pracę nad projektem. W celu zobrazowania harmonogramu, opracowano wykres Gantta (rys. 4.1), który przedstawia kolejne fazy projektu oraz przypisane im ramy czasowe.



Rys. 4.1. Wykres Gantta REPAIRO

4.2. System kontroli wersji i repozytorium projektu

W trakcie prac nad aplikacją wykorzystano system kontroli wersji **Git**, który umożliwił efektywne zarządzanie zmianami w kodzie. Cały projekt — kod źródłowy, pliki konfiguracyjne, struktura bazy danych oraz dokumentacja — został umieszczony na platformie **GitHub**.

<https://github.com/vLisek/SystemZarzadzaniaSerwisemSamochodowym>

Repozytorium zawiera ostateczną wersję projektu, okumentację projektowa oraz bazę danych. Zgodnie z wymaganiami, projekt dostępny będzie przez cały rok od daty oddania.

5. Prezentacja warstwy użytkowej projektu

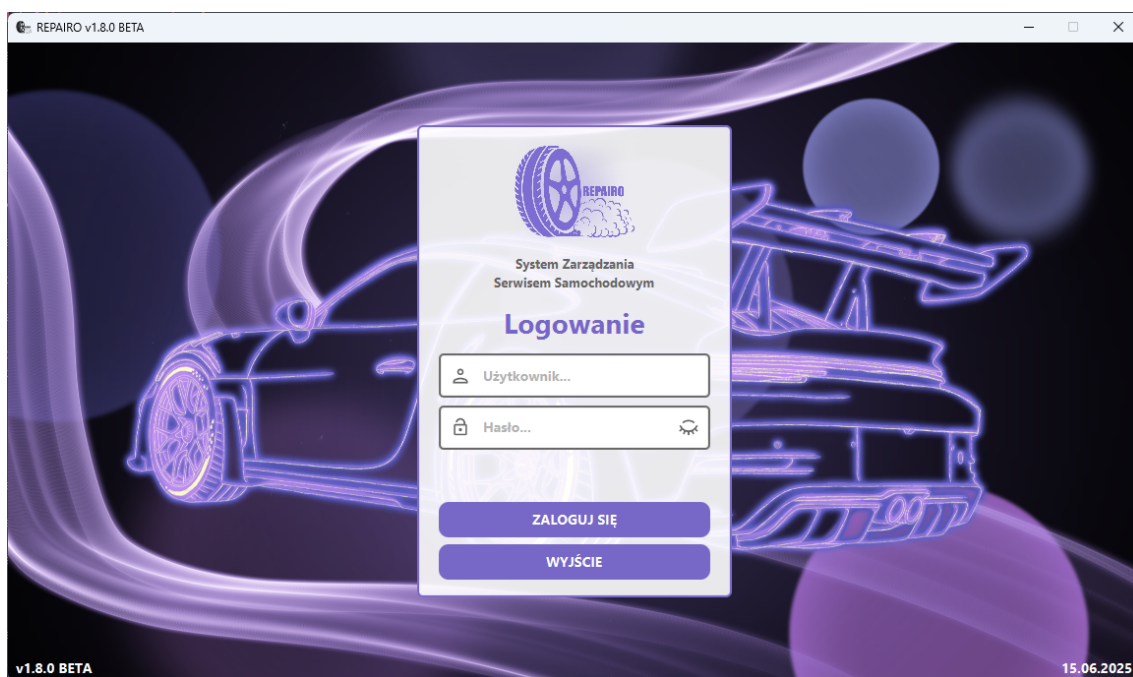
5.1. Opis szaty graficznej

Interfejs graficzny aplikacji został zaprojektowany z myślą o przejrzystości i intuicyjnej obsłudze. Zastosowano nowoczesną, minimalistyczną stylistykę z przewagą odcieni szarości, bieli i fioleto. Wszystkie ikony użyte w aplikacji pochodzą z serwisu **Icons8**, natomiast tło oraz logo zostały przygotowane na zamówienie, specjalnie na potrzeby projektu.

5.2. Prezentacja głównych widoków interfejsu użytkownika

5.2.1. Okno logowania

Ekran logowania jest pierwszym widokiem wyświetlanym użytkownikowi po uruchomieniu aplikacji. Składa się z formularza zawierającego pola do wpisania nazwy użytkownika i hasła, przycisku logowania oraz opcjonalnego komunikatu błędu w przypadku nieprawidłowych danych. Widok jest klarowny, aby nie rozpraszać użytkownika i umożliwić szybkie przejście do dalszej pracy.



Rys. 5.1. Okno logowania

5.2.4. Okno zarządzania klientami

Panel pozwala na zarządzanie danymi kontaktowymi klientów oraz ich typami (np. osoba prywatna lub firma). Można tu również dodać informację takie jak numer telefonu czy adres e-mail, co ułatwia kontakt z klientami w przyszłości.

Rys. 5.4. Okno zarządzania klientami

5.2.5. Okno zarządzania przyjęciami

W tym widoku możliwe jest tworzenie nowych zleceń serwisowych oraz przypisywanie ich do klientów, pojazdów oraz pracowników. Dodatkowo, użytkownik może dopisać rodzaj oraz ilość użytych części.

Rys. 5.5. Okno zarządzania przyjęciami

5.2.6. Okno zarządzania częściami

Użytkownik ma możliwość dodawania i usuwania części zamiennych dostępnych w warsztacie. Ponadto można przeglądać ich stany magazynowe oraz przypisane do nich ceny.

Rys. 5.6. Okno zarządzania częściami

5.2.7. Okno zarządzania wizytami

Panel prezentuje wizyty, daty zakończenia oraz aktualne statusy realizacji zleceń. Dodatkowo, można przemieszczać się pomiędzy miesiącami, aby przeglądać wizyty z różnych okresów. Widok ten jest kluczowy dla monitorowania bieżących i przyszłych zleceń w warsztacie. Na kolor czerwony zaznaczono termin oddania pojazdu. Na kolor niebieski zaznaczono aktualny dzień tygodnia.

Rys. 5.7. Okno zarządzania wizytami

5.2.8. Okno zarządzania pracownikami

Umożliwia jedynie wyświetlanie listy pracowników (na potrzeby projektu założono, że zarządzanie pracownikami odbywa się poprzez system bazodanowy).

ID	Imię	Nazwisko	Numer telefonu	Stanowisko	Nazwa użytkownika
1	Filip	Lisowski	534222323	Właściciel	flisowski
2	Konrad	Zaremba	507333444	Prezes zarządu	kzaremba
3	Ełzbieta	Leszczyńska	507444555	Dyrektor generalna	elieszczyńska
4	Ewelina	Bąk	506111222	Kierownik warsztatu	ebak
5	Tadeusz	Nowicki	506222333	Zastępca kierownika	tnowicki
6	Natalia	Grabowska	506333444	Kierownik ds. jakości	ngrabowska
7	Andrzej	Kruk	506444555	Kierownik zmiany	akruk
8	Katarzyna	Mazur	506555666	Recepcjonistka główna	kmazur
9	Barbara	Sadowska	506666777	Specjalistka ds. obsługi klienta	bsadowska
10	Michał	Dąbrowski	555333444	Mechanik	mdabrowski
11	Łukasz	Zieliński	504222333	Mechanik	tzeliński
12	Tomasz	Majewski	505111222	Diagnosta samochodowy	tmajewski
13	Kamil	Szymański	505222333	Blacharz	kszymanski
14	Paweł	Wójcik	505333444	Lakiernik	pwojick

Rys. 5.8. Okno zarządzania pracownikami

5.2.9. Okno zarządzania usługami

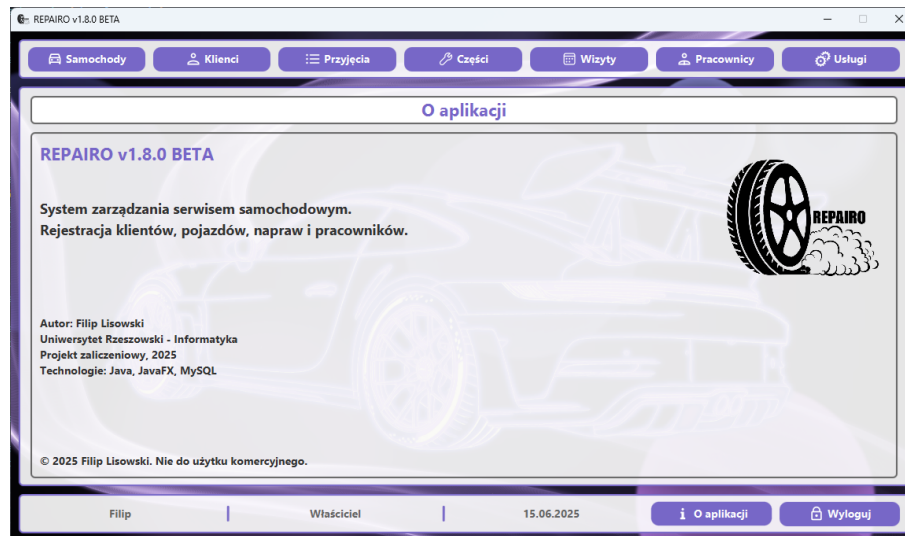
Widok służy do dodawania i modyfikowania usług oferowanych przez warsztat, wraz z przypisanym kosztem.

Nazwa usługi	Koszt usługi	Czas trwania usługi (godz.)
Wymiana klocków hamulcowych	240.0	2
Wymiana rozrządu	475.0	5
Lakierowanie	2500.0	12

Rys. 5.9. Okno zarządzania usługami

5.2.10. Okno wyświetlania informacji o aplikacji

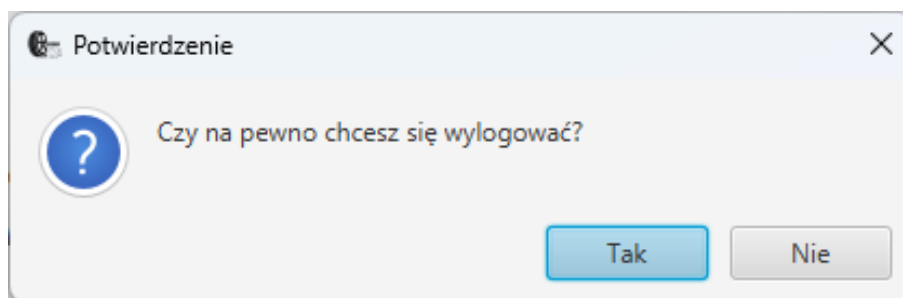
W tym miejscu użytkownik znajdzie informacje o wersji systemu, autorze projektu oraz danych kontaktowych.



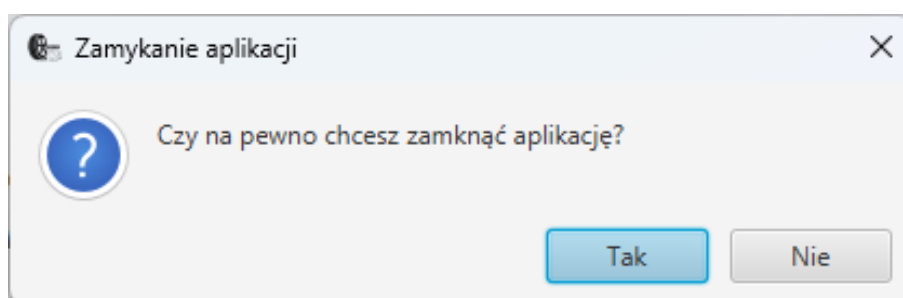
Rys. 5.10. Okno informacji o aplikacji

5.3. Mechanizm alertów systemowych

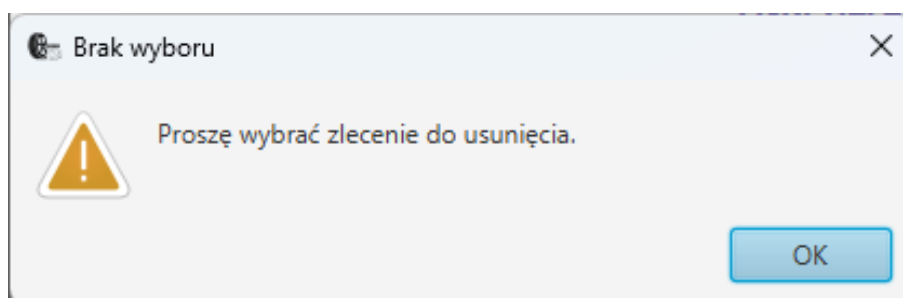
W aplikacji zastosowano dedykowany mechanizm powiadomień systemowych, który informuje użytkownika o sukcesie lub błędzie operacji. Powiadomienia te wyświetlane są w formie wyskakujących okien typu *popup*, zawierających ikonę, krótki komunikat. Poniżej przedstawiono przykładowe komunikaty wyświetlane użytkownikowi. Informują one o konieczności potwierdzenia wylogowania lub zamknięcia aplikacji, a także ostrzegają o błędnym wprowadzeniu danych.



Rys. 5.11. Alert potwierdzenia wylogowania



Rys. 5.12. Alert potwierdzenia wyjścia z aplikacji



Rys. 5.13. Alert o błędnym wprowadzeniu danych

Mechanizm ten został zaimplementowany w sposób uniwersalny, dzięki czemu może być łatwo wykorzystywany w dowolnym komponencie aplikacji.

6. Podsumowanie

W ramach niniejszego projektu zrealizowano system wspomagający zarządzanie serwisem samochodowym. Aplikacja została stworzona z wykorzystaniem języka Java oraz biblioteki JavaFX, a do komunikacji z relacyjną bazą danych użyto JDBC. Projekt obejmuje zarówno część interfejsu użytkownika, jak i warstwę logiki biznesowej oraz integrację z bazą danych zaprojektowaną zgodnie z zasadami normalizacji.

Główne funkcjonalności systemu to:

- zarządzanie klientami (dodawanie, edycja, usuwanie, przeglądanie danych),
- rejestracja i przegląd pojazdów wraz z ich szczegółowymi parametrami,
- ewidencja pracowników wraz z przypisanymi rolami i danymi logowania,
- obsługa zleceń serwisowych obejmujących klientów, pojazdy, pracowników, usługi, części oraz terminy wykonania,
- dynamiczne formularze z walidacją danych wejściowych oraz rozwijanymi listami powiązаныmi z danymi słownikowymi (np. typy nadwozia, rodzaje paliwa),
- automatyczne odświeżanie tabel po wykonaniu operacji CRUD.

System został zaprojektowany modularnie, co umożliwia jego dalszy rozwój i łatwe wprowadzanie nowych funkcjonalności. W ramach potencjalnych kierunków rozwoju projektu można wymienić:

- implementację systemu powiadomień (e-mail, SMS) o statusie zlecenia lub przypomnieniach dla klientów,
- integrację z zewnętrznymi systemami płatności oraz fakturowania,
- wdrożenie logowania za pomocą konta użytkownika z podziałem na role i poziomy uprawnień,
- rozszerzenie bazy danych o historię napraw i archiwum zleceń zakończonych,
- opracowanie wersji aplikacji internetowej dostępnej dla klientów,
- stworzenie panelu analitycznego prezentującego dane statystyczne dotyczące serwisu (ilość zleceń, przychody, najczęściej naprawiane modele pojazdów).

Zaprojektowany system REPAIRO stanowi solidną podstawę do dalszego rozwoju i może być z powodzeniem zastosowany w rzeczywistych warunkach działania małego lub średniego warsztatu samochodowego. Jego elastyczna architektura oraz przejrzysty interfejs użytkownika pozwalają na łatwe dostosowanie do indywidualnych potrzeb konkretnego serwisu.

7. Instrukcja uruchomienia aplikacji REPAIRO

7.0.1. Wymagania środowiskowe

Do poprawnego uruchomienia aplikacji REPAIRO wymagane jest przygotowanie środowiska zgodnie z poniższymi specyfikacjami programowymi:

- System operacyjny: **Windows 10** lub nowszy
- Środowisko JDK: **OpenJDK 23.0.2**
- IDE: **IntelliJ IDEA Community Edition 2024.3.3**
- Sterownik JDBC: **mysql-connector-java-8.0.33** dołączony do repozytorium w katalogu `mysql-connector-j-9.3.0`.
- Baza danych: **MariaDB** (środowisko **phpMyAdmin 5.2.1**)
- Panel sterowania: **XAMPP Control Panel**

Uwaga: Domyślnym użytkownikiem serwera bazy danych jest `root`, a hasło puste (`"`), zgodnie z domyślną konfiguracją XAMPP.

7.0.2. Baza danych

- Do repozytorium dołączony jest plik `warsztat.sql`, który zawiera strukturę i przykładowe dane bazy danych.
- Po uruchomieniu serwera MySQL należy zaimportować bazę danych do środowiska phpMyAdmin.
- Domyślny użytkownik: `root`, Hasło: (puste lub zgodne z konfiguracją lokalną)
- Nazwa bazy danych: `warsztat`

7.0.3. Konfiguracja połączenia z bazą danych

W katalogu głównym projektu znajduje się plik:

```
config-template.properties
```

Zawiera on domyślną konfigurację połączenia z bazą danych i **musi zostać skopiowany oraz dostosowany** przed pierwszym uruchomieniem aplikacji.

Kroki:

1. Skopiuj plik:

```
config-template.properties → config.properties
```

2. Otwórz `config.properties` w edytorze tekstu.

3. Uzupełnij dane zgodnie z lokalną konfiguracją użytkownika bazy danych:

```
db.url=jdbc:mysql://localhost:3306/warsztat
db.user=root
db.password=
```

Uwaga: Klasa `DatabaseConnector` pobiera dane z tego pliku w celu nawiązania połączenia z bazą. Dzięki temu nie trzeba edytować kodu źródłowego przy każdej zmianie konfiguracji.

7.0.4. Kroki uruchomienia aplikacji REPAIRO

1. Klonowanie projektu z GitHub

- (a) Przejdź do repozytorium projektu:

<https://github.com/vLisek/SystemZarzadzaniaSerwisemSamochodowym.git>

- (b) Pobierz repozytorium lokalnie:

(c) `git clone https://github.com/vLisek/SystemZarzadzaniaSerwisemSamochodowym.git`

- (d) Otwórz folder projektu w IntelliJ IDEA Community Edition.

2. Dodanie sterownika JDBC

- (a) Umieść folder `mysql-connector-j-9.3.0` w katalogu projektu

- (b) Dodaj go jako bibliotekę w: `File > Project Structure > Modules > Dependencies`

3. Import bazy danych

- (a) Uruchom XAMPP Control Panel i włącz moduł MySQL

- (b) Otwórz phpMyAdmin w przeglądarce: *<http://localhost/phpmyadmin>*

- (c) Utwórz nową bazę danych: `warsztat`

- (d) Zaimportuj plik `warsztat.sql`

4. Uruchomienie aplikacji

- (a) Kliknij przycisk Run w klasie `Main.java`

- (b) Aplikacja otworzy ekran logowania

7.0.5. Dane testowe (logowanie)

- **Administator**

- Nazwa użytkownika: admin
- Hasło: admin

- **Mechanik**

- Nazwa użytkownika: kszymanski
- Hasło: 1234

Po wykonaniu powyższych kroków środowisko jest gotowe do pracy z aplikacją REPAIRO.

Spis rysunków

3.1	Struktura katalogów projektu	16
3.2	Struktura pakietów	17
3.3	Diagram UML	19
3.4	Diagram ERD bazy danych warsztat	21
4.1	Wykres Gantta REPAIRO	23
5.1	Okno logowania	24
5.2	Główne okno interfejsu	25
5.3	Okno zarządzania samochodami	25
5.4	Okno zarządzania klientami	26
5.5	Okno zarządzania przyjęciami	26
5.6	Okno zarządzania częściami	27
5.7	Okno zarządzania wizytami	27
5.8	Okno zarządzania pracownikami	28
5.9	Okno zarządzania usługami	28
5.10	Okno informacji o aplikacji	29
5.11	Alert potwierdzenia wylogowania	30
5.12	Alert potwierdzenia wyjścia z aplikacji	30
5.13	Alert o błędnym wprowadzeniu danych	30

Spis tabel

2.1	Dostęp poszczególnych ról użytkowników do modułów aplikacji.	11
3.1	Pliki FXML i ich przeznaczenie	20

Załącznik nr 2 do Zarządzenia nr 228/2021 Rektora Uniwersytetu Rzeszowskiego z dnia 1 grudnia 2021 roku w sprawie ustalenia procedury antyplagiatowej w Uniwersytecie Rzeszowskim

OŚWIADCZENIE STUDENTA O SAMODZIELNOŚCI PRACY

..... Filip Lisowski
Imię (imiona) i nazwisko studenta

Wydział Nauk Ścisłych i Technicznych

..... Informatyka
Nazwa kierunku

..... 134942
Numer albumu

1. Oświadczam, że moja praca projektowa pt.: Aplikacja REPAIRO - dokumentacja projektowa
 - 1) została przygotowana przeze mnie samodzielnie*,
 - 2) nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (t.j. Dz.U. z 2021 r., poz. 1062) oraz dóbr osobistych chronionych prawem cywilnym,
 - 3) nie zawiera danych i informacji, które uzyskałem/am w sposób niedozwolony,
 - 4) nie była podstawą otrzymania oceny z innego przedmiotu na uczelni wyższej ani mnie, ani innej osobie.
2. Jednocześnie wyrażam zgodę/~~nie wyrażam zgody~~** na udostępnienie mojej pracy projektowej do celów naukowo-badawczych z poszanowaniem przepisów ustawy o prawie autorskim i prawach pokrewnych.

Rzeszów, 27.06.2025
(miejscowość, data)

Filip Lisowski
(czytelny podpis studenta)

* Uwzględniając merytoryczny wkład prowadzącego przedmiot

** – niepotrzebne skreślić